

Airbnb Price Prediction -- ML Project Report

Student: Elad Shoham

I.D: 205439649

Email: elads361@gmail.com

1. Introduction

In this project I built a model to predict the nightly price of Airbnb listings in New York City. The question is simple: given basic facts about an apartment -- its size, location, and room type -- what price should it cost per night?

This is useful for hosts who want to set a fair price, and for guests who want to know if a listing is overpriced.

The project also demonstrates key concepts from the course: cross-validation, bias-variance tradeoff, feature correlation, log transformation, regularisation, and residual analysis.

2. The Data

Source: Inside Airbnb (insideairbnb.com) -- New York City, 2024

Listings before cleaning: 35,036

Listings after cleaning: 19,859

I removed listings with a price of \$0 or above \$2,000, since those are extreme values that do not represent typical rentals.

I used 12 features:

- neighbourhood -- which area of NYC
- room_type -- entire home, private room, hotel, or shared room
- bedrooms / beds / bathrooms -- apartment size
- accommodates -- how many guests it fits
- review_scores_rating -- average guest rating
- number_of_reviews -- how many reviews the listing has
- minimum_nights -- shortest allowed stay
- latitude / longitude -- exact location
- host_is_superhost -- whether the host has superhost status

Some columns had missing values. For bedrooms, beds, bathrooms, and review score I filled the missing values with the median. This is called median imputation.

Figure 1: Price Distribution (original)

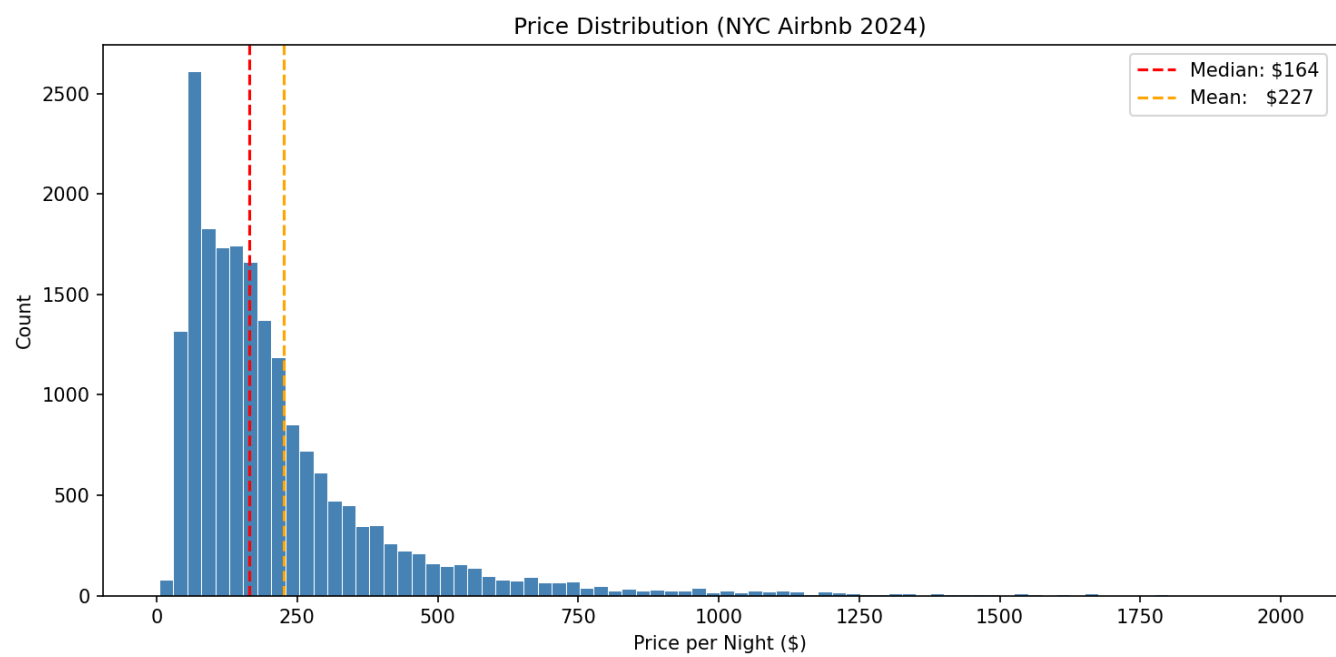


Figure 2: Log-Price Distribution (after transformation)

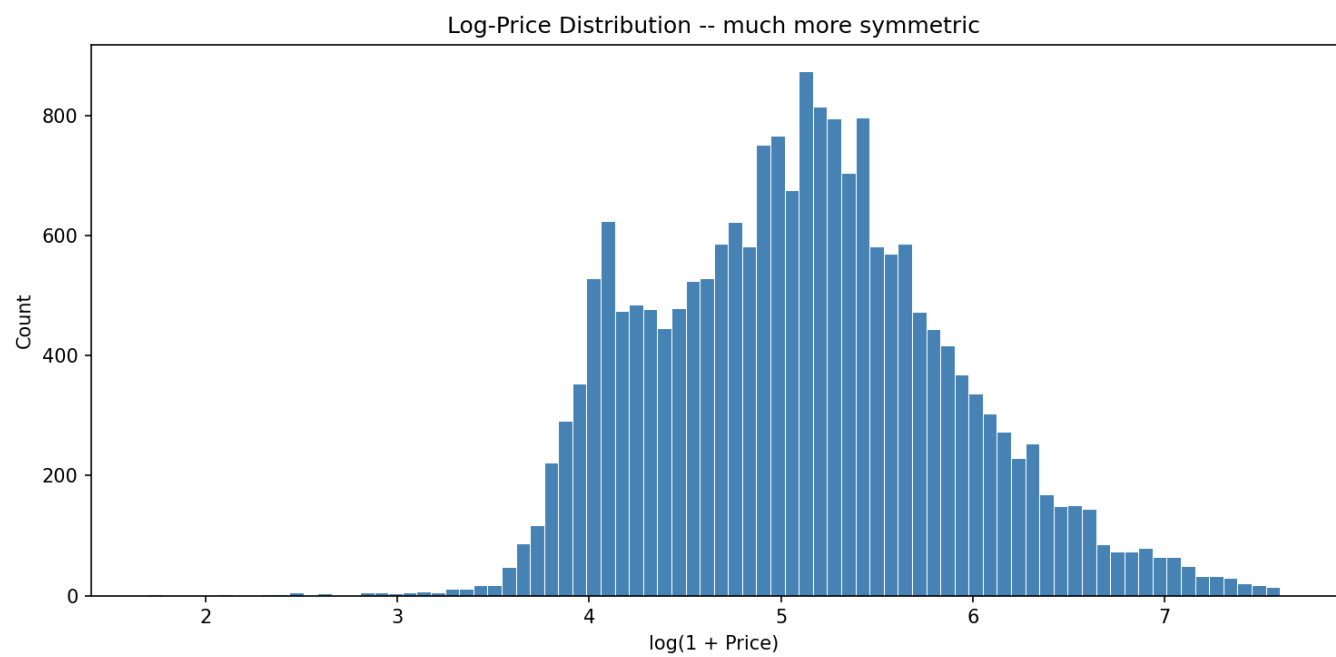


Figure 3: Median Price by Room Type

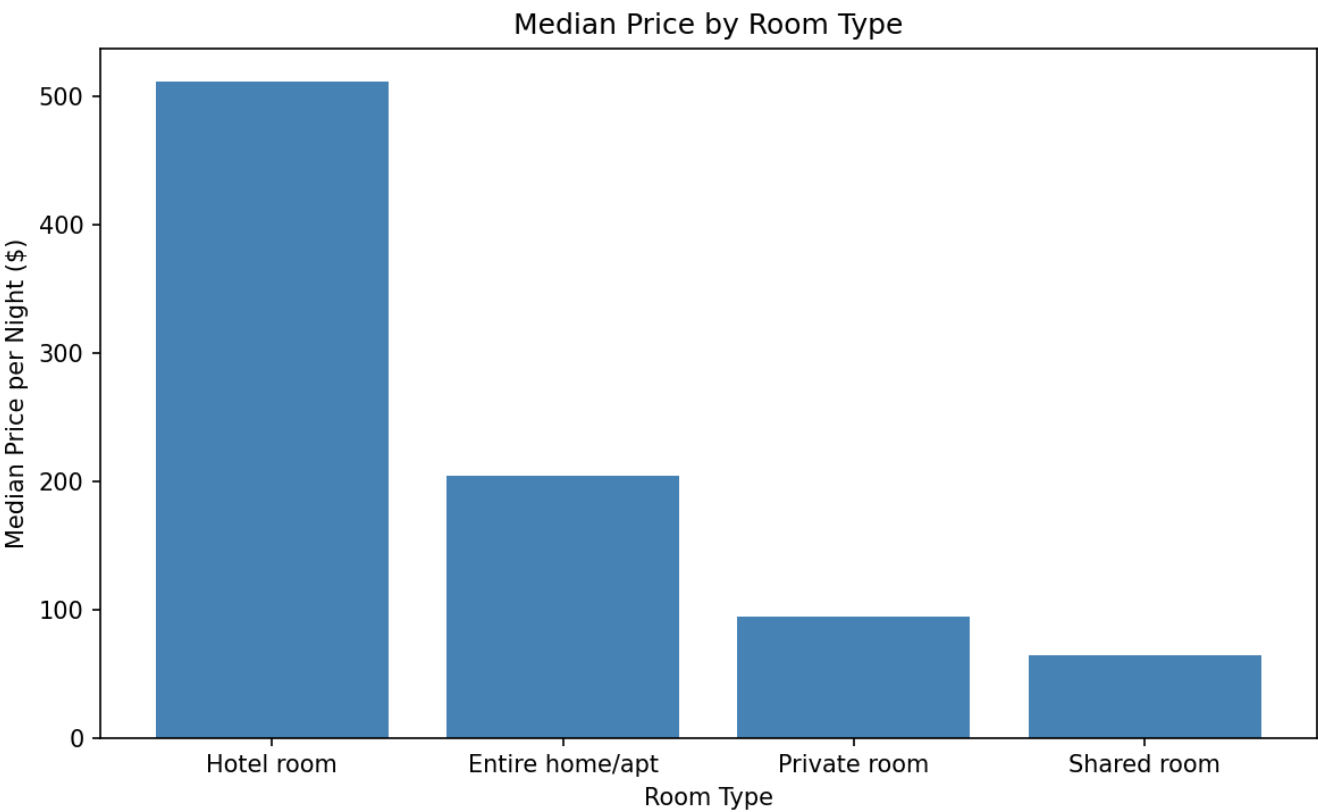


Figure 4: Top 15 Neighbourhoods by Price



Airbnb Price Prediction -- ML Project Report

Figure 5: Geographic Map of Listings (colour = price)

NYC Airbnb Listings -- colour = price

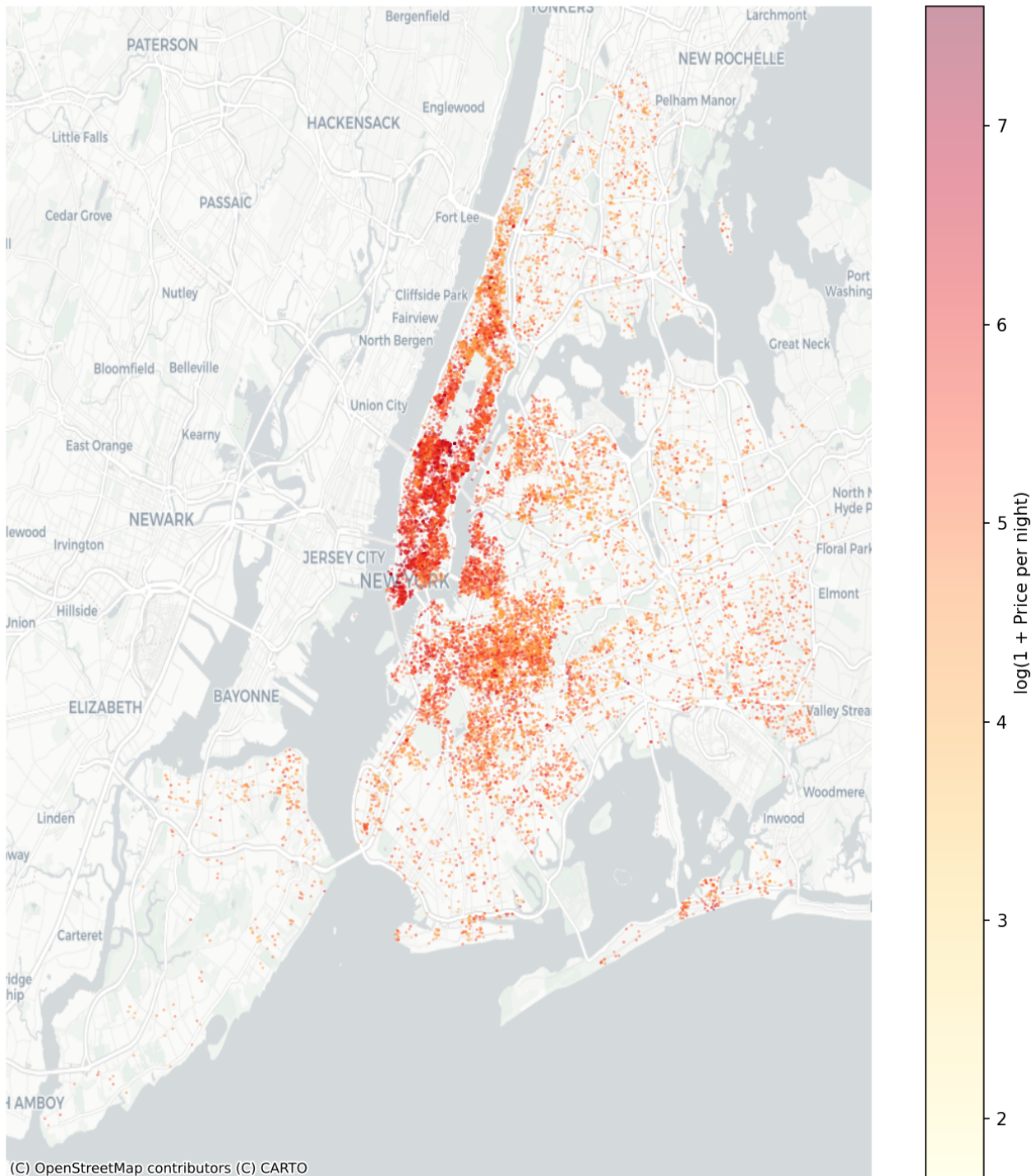
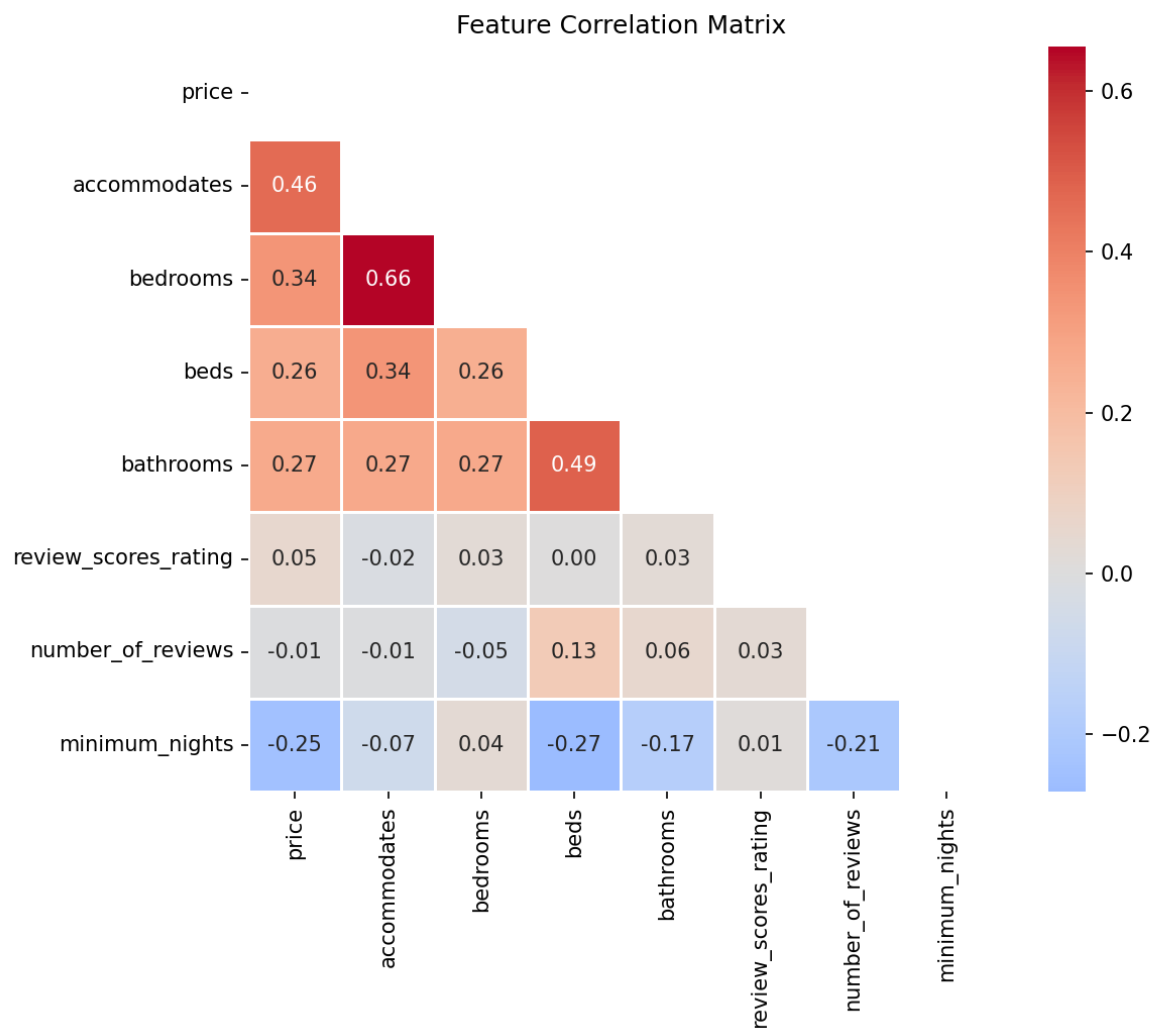
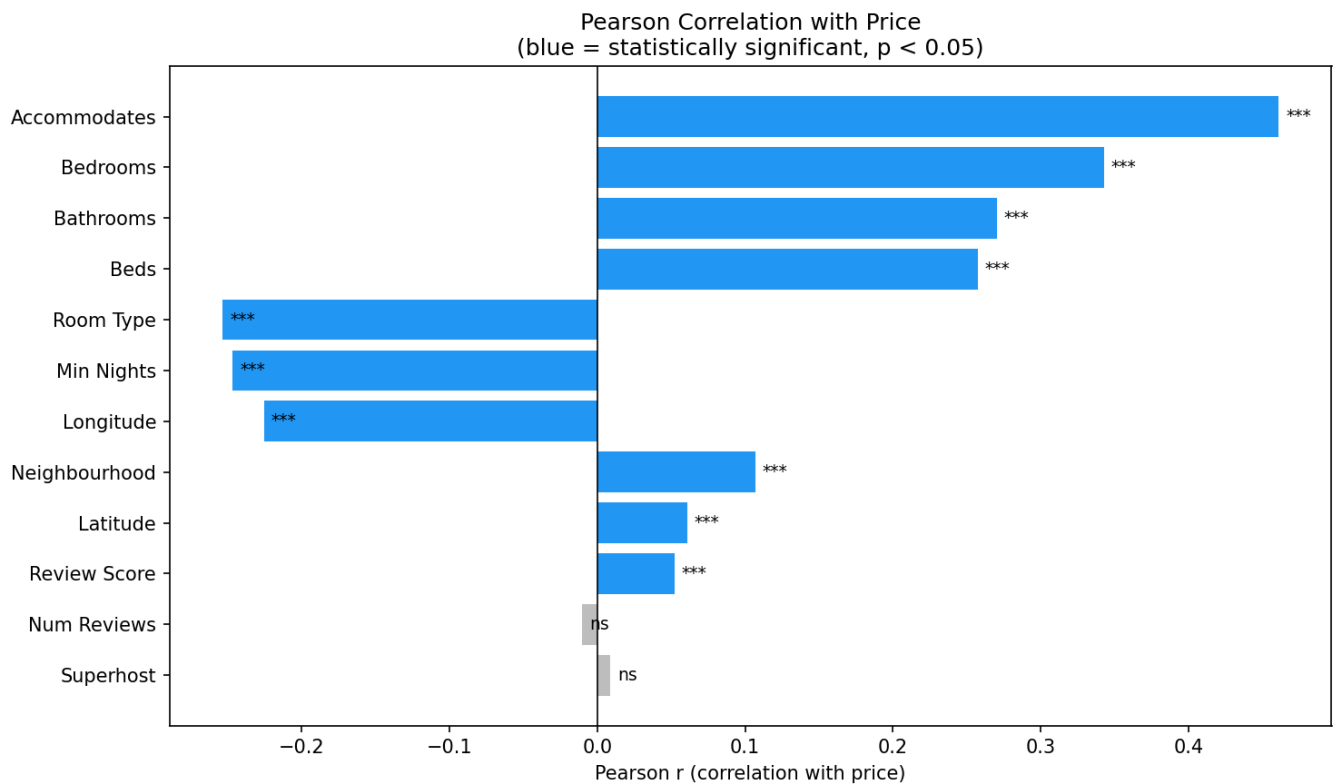


Figure 6: Feature Correlation Matrix



Airbnb Price Prediction -- ML Project Report

Figure 7: Pearson Correlation with Price (scipy.stats.pearsonr)



3. Looking at the Data

Average price: \$227 per night

Median price: \$164 per night

The mean is higher than the median because a small number of very expensive listings pull the average up. The median is a better description of what a typical listing costs.

The map (Figure 5) shows that Manhattan listings are the most expensive. This is why latitude and longitude are useful -- they tell the model exactly where the listing is, which is more precise than just the neighbourhood name.

The correlation matrix (Figure 6) shows that accommodates, bedrooms, beds, and bathrooms are all strongly correlated. They all measure apartment size. This is called multicollinearity. It means the model splits importance between these features, so each one looks less important than it really is. Together they represent one idea: how big the apartment is.

Review score has a weak correlation with price. In NYC, landlords can charge high prices regardless of rating because supply is limited.

Figure 7 shows the Pearson correlation (r) between each feature and price, computed with `scipy.stats.pearsonr`. This returns two values: r (the strength of the correlation) and a p -value (how likely this correlation appeared by random chance). Features marked * have $p < 0.05$, which means the result is statistically significant. Features marked 'ns' are not significant.

Accommodates and bedrooms have the strongest correlation with price ($p < 0.001$). Latitude and longitude are also significant -- location matters a lot. Review score and minimum nights are close to zero and not significant.

4. Log Transformation

Figure 1 shows that prices are skewed to the right -- most listings are cheap, but a few are very expensive. When the target is skewed, models waste effort trying to predict the extreme values, which hurts overall accuracy.

The fix is to train on $\log(1 + \text{price})$ instead of price. Figure 2 shows the result: the distribution is now more symmetric and bell-shaped, which is much easier for a model to learn.

Airbnb Price Prediction -- ML Project Report

After the model makes a prediction in log scale, I convert back to dollars using $\exp(\text{prediction}) - 1$. The final output is always in dollars.

5. Preparing the Data

Categorical columns (neighbourhood, room type) were converted to numbers using label encoding.

Linear models are sensitive to the scale of features. A feature with large values (like number of reviews) can dominate a feature with small values (like superhost status), even if the small one is more important. Tree models do not have this problem.

To make the comparison fair, I used StandardScaler before the linear models inside a sklearn Pipeline. The scaler learns the mean and standard deviation only from training data, which prevents data leakage.

The data was split: 80% for training, 20% for testing. The test set was not used until the very end.

6. The Models

I tested three models. Each one learns differently, and each connects to a topic from the course.

--- Linear Regression ---

Linear Regression predicts price as a weighted sum of the features:

$$\text{price} = a * \text{bedrooms} + b * \text{latitude} + c * \text{accommodates} + \dots$$

The model finds the weights (a, b, c, ...) that minimise the total squared error on the training data. It is the simplest model. It works well when the relationship between features and price is roughly linear. The problem is that real pricing is rarely a straight line.

--- Ridge Regression (Tikhonov Regularisation / SRM) ---

Ridge is like Linear Regression, but with an extra penalty in the objective:

$$\text{minimise: training error} + \alpha * \text{sum of squared coefficients}$$

The penalty forces the weights to stay small, which stops the model from overfitting. This is called L2 regularisation, or Tikhonov regularisation.

This connects to Structural Risk Minimisation (SRM): the penalty term limits how complex the model is allowed to be. A larger alpha means a simpler model -- less variance (less overfitting) but more bias (less flexible). The VC dimension of a linear model with d features is d+1. Here d=12, so VC dim = 13. This small hypothesis class explains why linear models show high bias in the learning curves -- they are not flexible enough to learn the non-linear structure of Airbnb prices.

Another benefit: when features are correlated (bedrooms, beds, accommodates), plain Linear Regression can give them large and unstable weights. Ridge spreads the weights more evenly, which makes them easier to interpret.

--- Gradient Boosting ---

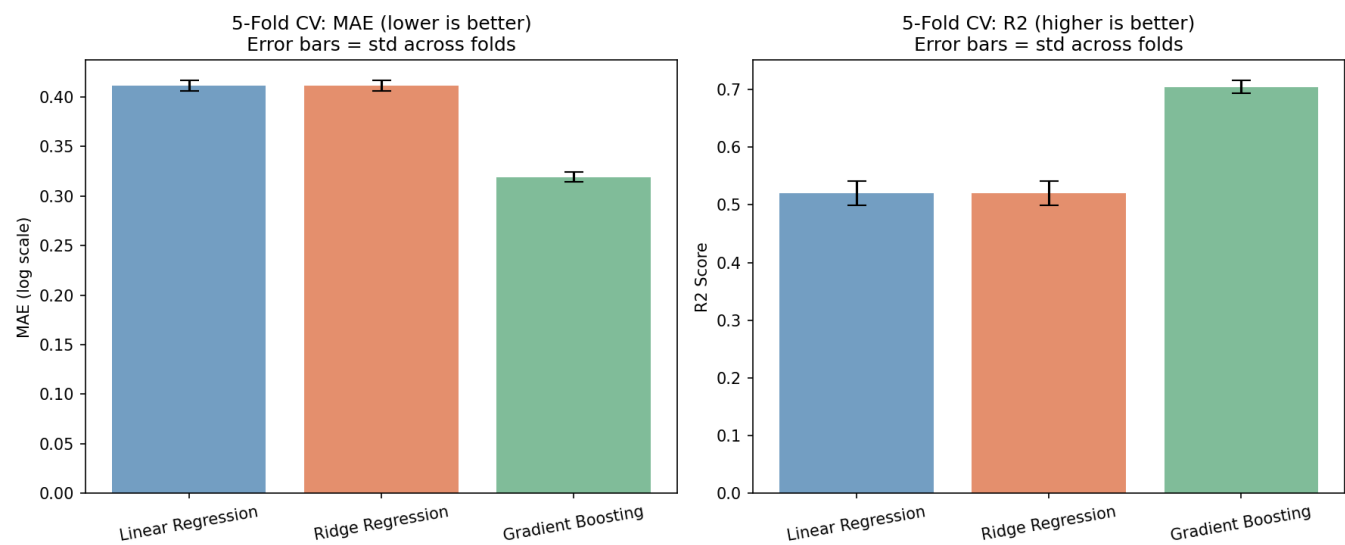
Gradient Boosting builds many small decision trees, one after another. Each new tree learns to fix the mistakes of the previous ones. More precisely: each tree is trained on the residuals (errors) of the current model. The final prediction is the sum of all trees.

This is related to AdaBoost from the course. Both are boosting methods that combine weak learners into a strong one. The difference is that Gradient Boosting can minimise any differentiable loss function (here: squared error on log-price), while AdaBoost was designed for classification with exponential loss.

Airbnb Price Prediction -- ML Project Report

Gradient Boosting has a much larger effective VC dimension than linear models. This means it can learn more complex patterns -- but also needs regularisation (`max_depth`, `learning_rate`) to avoid overfitting.

Figure 8: 5-Fold Cross-Validation Results



7. Comparing Models with Cross-Validation

Instead of one train/test split, I used 5-fold cross-validation. The training data is divided into 5 equal parts. Each time, 4 parts are used for training and 1 for testing. This rotates through all 5 combinations, giving 5 error estimates. The mean shows how well the model performs, and the standard deviation shows how stable it is.

By the Central Limit Theorem (CLT), the mean of many independent error estimates converges to a normal distribution, no matter what shape the individual errors have. This makes the mean CV score a reliable estimate of true performance, and lets us build confidence intervals around it.

The No Free Lunch theorem says that no model is better than all others on every problem. This is why comparing multiple models is necessary -- we cannot know in advance which one fits this specific dataset.

Results:

Linear Regression MAE = 0.4110 (+/- 0.0051) R2 = 0.520 (+/- 0.021)

Ridge Regression MAE = 0.4110 (+/- 0.0051) R2 = 0.520 (+/- 0.021)

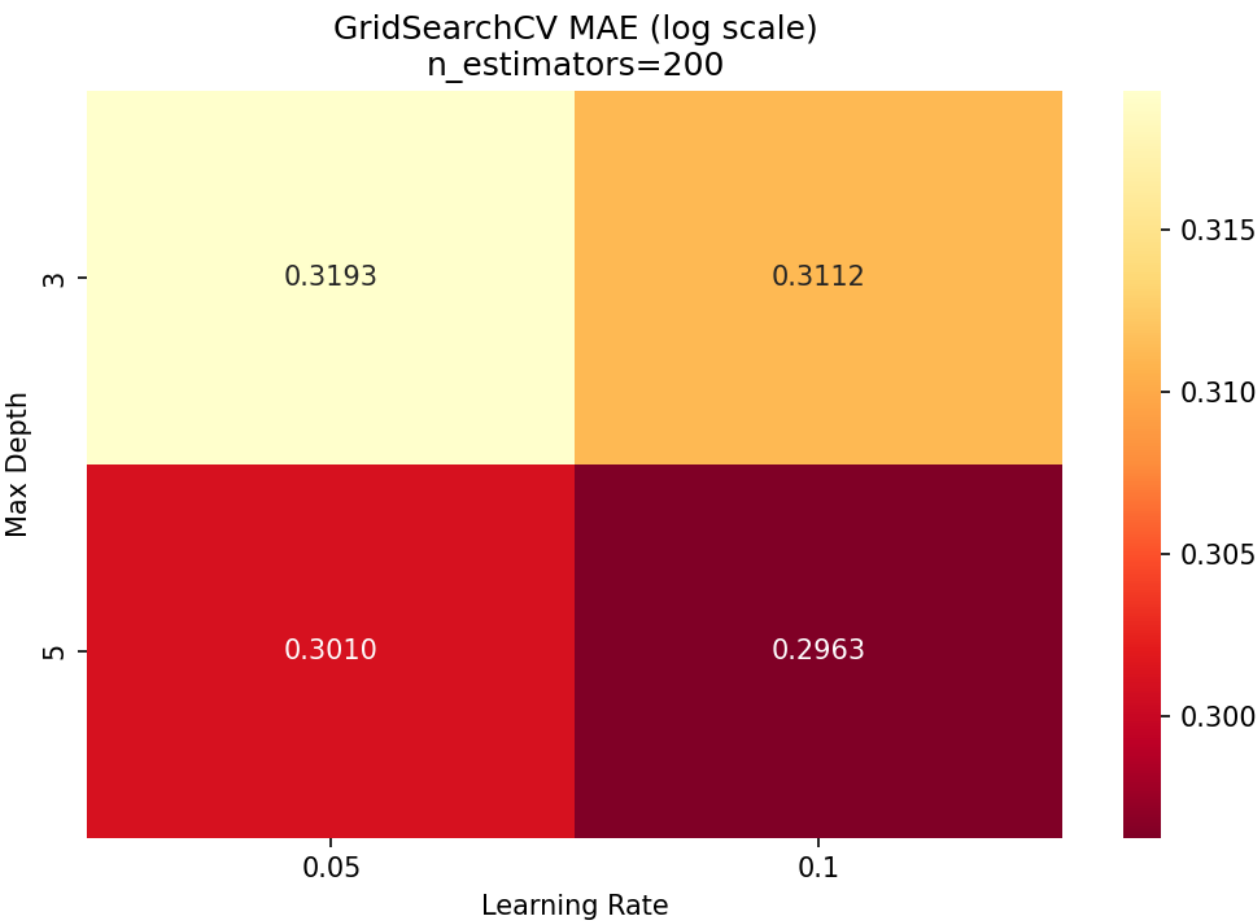
Gradient Boosting MAE = 0.3188 (+/- 0.0050) R2 = 0.704 (+/- 0.011)

Ridge and Linear Regression give almost the same result. This makes sense: regularisation helps most when the number of features is large compared to the number of samples, or when features are very correlated. Here we have ~16,000 samples and only 12 features, so overfitting is not a big problem and the Ridge penalty barely changes anything.

Ridge still has a theoretical benefit: it gives more stable weights when features are correlated, which makes the model easier to interpret.

Gradient Boosting performs best. I used GridSearchCV to tune its settings.

Figure 9: GridSearchCV -- MAE by Hyperparameters



8. Hyperparameter Tuning with GridSearchCV

Every model has settings that are not learned from data -- they are chosen before training. These are called hyperparameters. Choosing them well can improve performance significantly.

For Gradient Boosting I tuned three hyperparameters:

- `n_estimators`: how many trees to build (100 or 200)
- `max_depth`: how deep each tree can grow (3 or 5)
- `learning_rate`: how much each tree contributes (0.05 or 0.1)

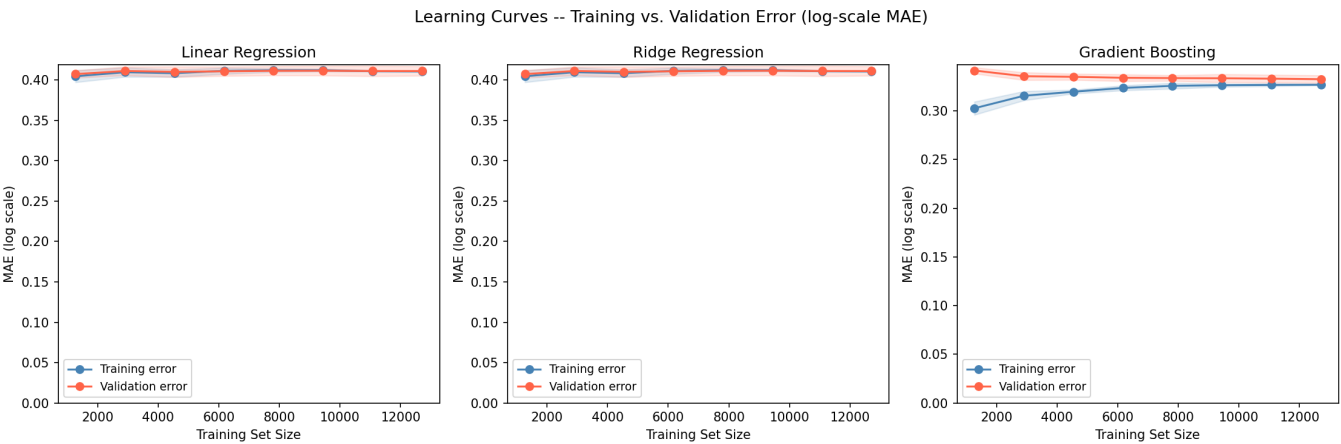
GridSearchCV tries all combinations (8 total) using 5-fold cross-validation and picks the best one automatically.

Best settings found:

```
n_estimators = 200
max_depth    = 5
learning_rate = 0.1
```

Figure 9 shows how MAE changes with different combinations. Deeper trees with a higher learning rate tend to overfit. Shallower trees with a lower learning rate tend to underfit.

Figure 10: Learning Curves -- Bias vs. Variance



9. Bias vs. Variance -- Learning Curves

A learning curve shows how training error and validation error change as we add more training data. It is a tool for diagnosing whether a model has too much bias or too much variance.

High bias (underfitting): both errors are high and close together. The model is too simple. Adding more data will not help.

High variance (overfitting): training error is much lower than validation error. The model memorises the training data. More data would help.

What Figure 10 shows:

Linear Regression: both errors converge quickly at a high level. This is high bias. The model is too simple for this problem.

Ridge Regression: very similar to Linear Regression. Both are linear models, so they have similar bias. Ridge reduces variance a little through regularisation, but the main problem is that no linear model can capture the non-linear structure of NYC pricing.

Gradient Boosting: lower error and a smaller gap between training and validation. A good balance between bias and variance.

Connection to PAC Learning: the Fundamental Theorem of PAC Learning says that generalisation error has two parts -- approximation error (bias: how well the best function in the class fits the target) and estimation error (variance: how much the result changes with different training data). The learning curves make this decomposition visible directly.

The gap between training and validation error is the empirical version of the PAC generalisation bound. A large gap means high variance. A small gap at a high error level means high bias.

All three curves reach a plateau, which means we have enough data. Adding more samples would not help much for these models.

Figure 11: Predicted vs Actual Price (Gradient Boosting, tuned)

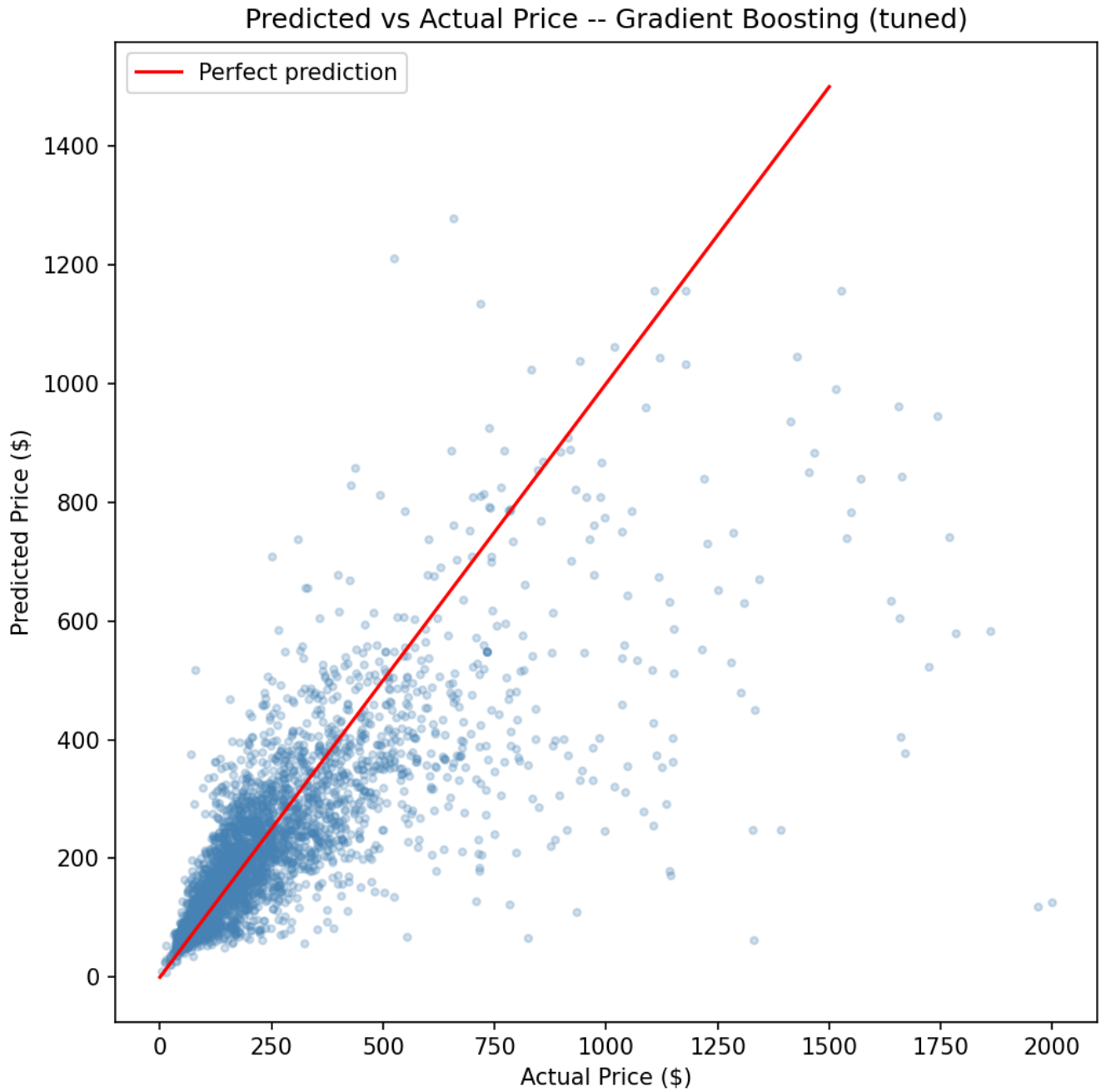
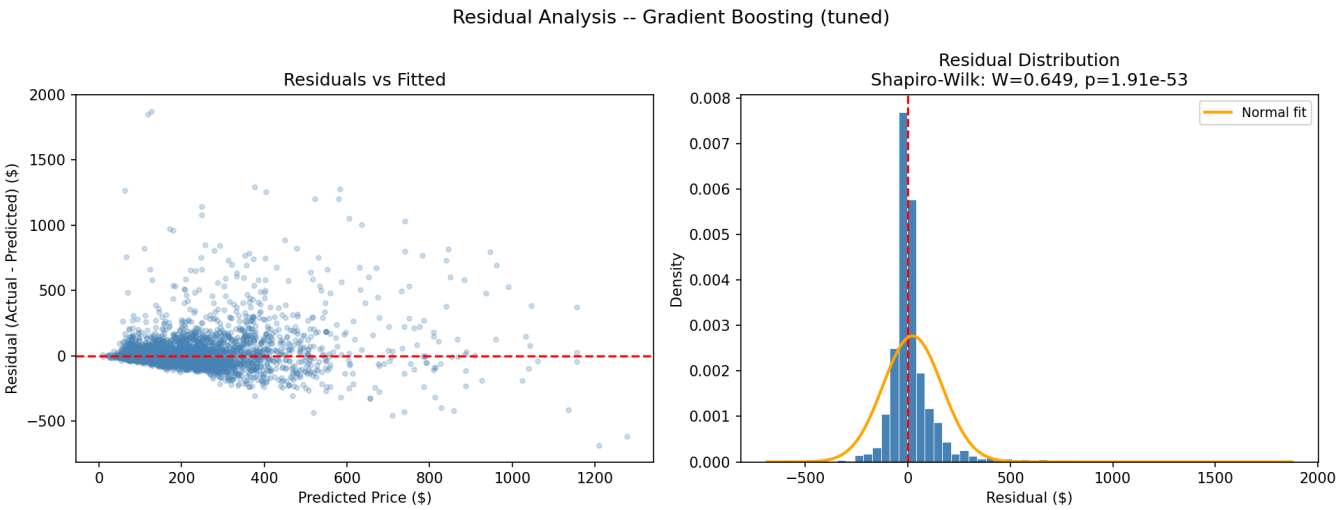


Figure 12: Residual Analysis + Shapiro-Wilk Test



10. Final Results and Residual Analysis

The tuned Gradient Boosting model was tested once on the held-out test set.

Test MAE: \$72 per night

Test R2: 0.570

These numbers are close to the cross-validation results, which means the model did not overfit.

The residual is the difference between the actual price and the predicted price. In a perfect model, residuals would be randomly spread around zero.

The 'Residuals vs Fitted' plot (Figure 12, left) shows a fan shape at higher predicted prices. This is called heteroscedasticity -- the errors are larger for expensive listings. Luxury apartments are harder to predict because they depend on things not in the data, like interior quality or view.

The right plot shows the residual distribution with a fitted normal curve. To formally test whether the residuals are normally distributed, I used the Shapiro-Wilk test (`scipy.stats.shapiro`):

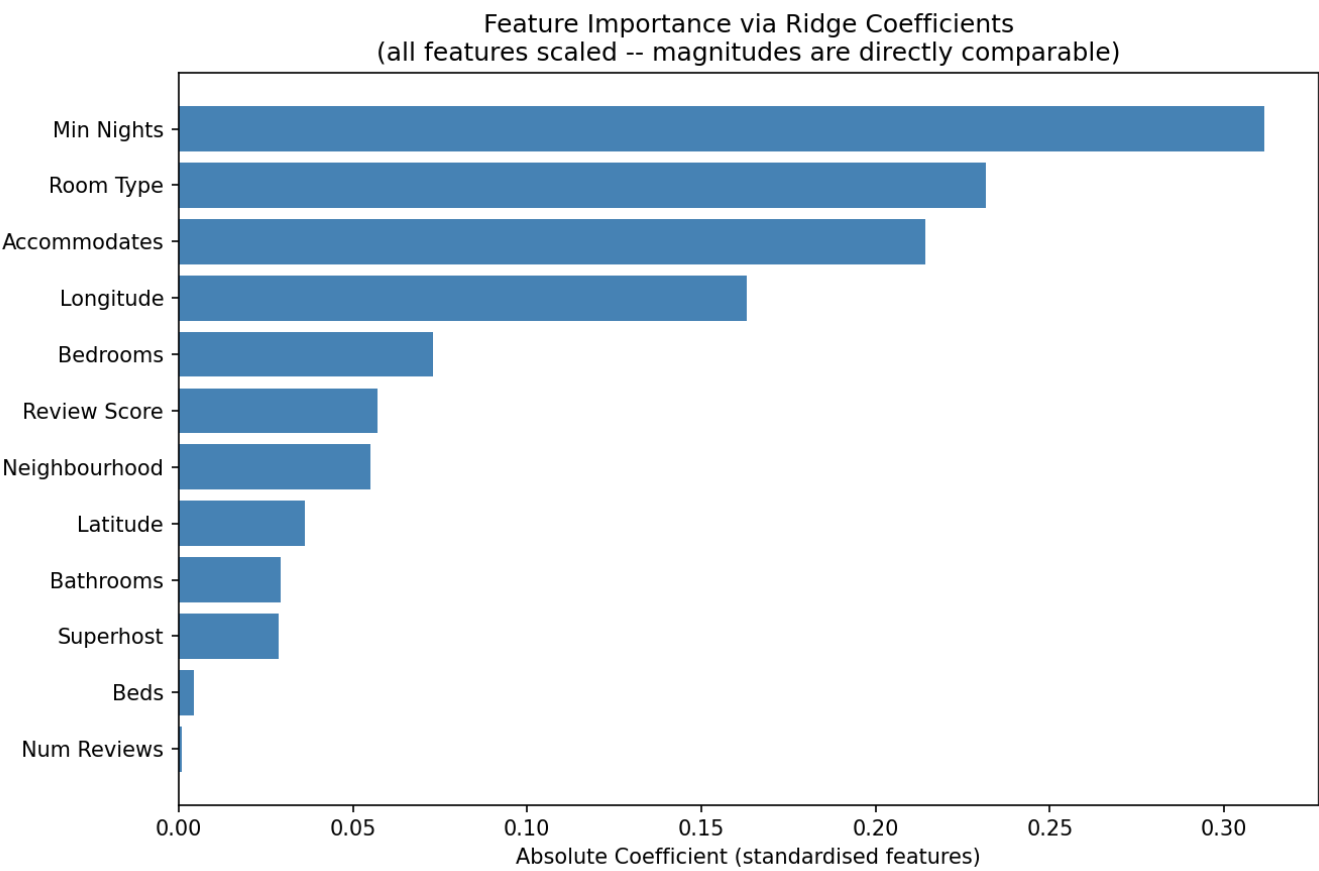
H0: the residuals follow a normal distribution

H1: the residuals do not follow a normal distribution

Result: $W = 0.6486$, $p = 1.91e-53$

The p-value is very small ($p < 0.05$), so we reject H0. The residuals are not perfectly normal. This is expected -- a few very expensive listings create a heavy right tail that a normal curve cannot capture. This confirms the model still struggles with luxury listings.

Figure 13: Feature Coefficients (Ridge Regression)



11. What Affects the Price Most?

Figure 13 shows the Ridge Regression coefficients after scaling all features with StandardScaler. Because every feature is on the same scale, the size of each coefficient directly shows how much that feature affects the price.

This is one advantage of Ridge over tree models: the coefficients are easy to interpret. In tree-based models, importance scores are harder to understand.

Latitude and longitude have the largest coefficients. This confirms that exact location is the strongest driver of price in NYC.

Bedrooms and accommodates also have large coefficients -- apartment size matters.

Superhost status and number of reviews have small coefficients. In NYC's competitive rental market, reputation matters less than location and size.

Note: bedrooms, beds, bathrooms, and accommodates are all correlated (see Figure 6). Ridge helps here by spreading the weights more evenly among correlated features, instead of giving one large arbitrary weight to just one of them.

12. Conclusion

I built a Gradient Boosting model to predict Airbnb prices in NYC using 19,859 listings.

Main lessons from this project:

- Log-transforming a skewed target makes it easier for the model to learn
- Latitude and longitude are strong features -- exact location matters more than just the neighbourhood name
- Cross-validation gives a more reliable result than a single train/test split
- GridSearchCV automates hyperparameter tuning and improves performance
- Learning curves show the bias-variance tradeoff clearly
- Residual analysis shows where the model still makes mistakes
- The No Free Lunch theorem justifies testing multiple models
- Ridge regression implements SRM by penalising model complexity

Final result: MAE = \$72 per night, R2 = 0.570